Reinforcement Learning for Block Blast

From Deep Q-Networks to Deep Value Networks

An Afterstate-Based Approach to Combinatorial Puzzle Solving

Nathan

March 2026

Contents

Abstract

We investigate reinforcement learning approaches to the puzzle game *Block Blast*, played on an 8×8 grid. The agent must place Tetromino-like pieces to clear full rows and columns. We study two environment variants: a single-piece variant (1P) where one piece is given at a time, and a three-piece variant (3P) where three pieces are presented simultaneously and must all be placed in a freely chosen order. We experiment with classical Deep Q-Network approaches (DDQN, Rainbow DQN) and introduce a *Deep Value Network* (DVN) that operates on *afterstates*—the board states resulting from each possible placement—instead of learning a standard Q-function. The DVN achieves significantly better performance than naive baselines with only 42,461 parameters, compared to the $\sim$2.67M parameters needed by DQN-based models. We describe the reward shaping decisions, the architectures, and the training procedure, and we report benchmarks against greedy and random baselines.

1 Introduction

Block Blast is a single-player puzzle game played on an 8×8 grid. At each turn, the player receives one or more Tetromino-like pieces and must place them on the grid. When a full row or column is formed, it is cleared and points are scored. The game ends when no more pieces can be placed. The challenge lies in planning placements that keep the board clean while maximising clearing combos.

This game presents several classic challenges for reinforcement learning:

- **Long-horizon planning**: clearing lines requires setting up the board over many turns.
- **Large action space**: up to 64 positions per piece in the 1P variant, 192 in the 3P variant.
- **Sparse rewards**: most individual placements yield no line clears and thus no large reward signal.
- **Combinatorial structure**: in the 3P variant, the order in which the three pieces are placed matters significantly.

We organised our experiments in three phases. First, we designed the environment and iterated on the reward function. Second, we trained standard DQN-based agents (DDQN and Rainbow DQN) directly mapping game state to Q-values. Third, we developed a *Deep Value Network* (DVN) that estimates the long-term value of a board position, combined with a one-step lookahead to select actions.

2 Environment

2.1 Block Blast: Game Rules

The game is played on an 8×8 binary grid (0 = empty, 1 = filled). Seven piece shapes are used, each randomly rotated by a multiple of 90 degrees at the start of each episode:

Shape	Description
Dot	1×1 single cell
Square	2×2 block
Big Square	3×3 block
Line-3	1×3 horizontal strip
Line-4	1×4 horizontal strip
L-small	2×2 L-shape
L-large	3×3 L-shape

After each placement, every row and column that is entirely filled is cleared simultaneously. The agent must avoid reaching a state where no valid placement exists, as this ends the episode.

2.2 Single-Piece Variant (1P)

In the 1P variant, one piece is drawn uniformly at random at the start of each step. The observation space is a dictionary containing:

- `board`: the 8×8 integer grid,
- `piece`: the piece shape encoded in a 5×5 padded grid,
- `valid_placements`: an 8×8 binary mask of valid top-left positions,
- `placements_result`: a precomputed $(8, 8, 8, 8)$ tensor giving the resulting board for every candidate placement, alongside an $(8, 8)$ reward map.

The action space is Discrete(64), where action $a = 8r + c$ places the piece with its top-left corner at row r, column c.

2.3 Three-Piece Variant (3P)

In the 3P variant, three pieces are drawn at the start of each *round*. The agent must place all three pieces in a freely chosen order before a new round begins. The action space becomes Discrete(192):

$$a = \text{piece_idx} \times 64 + r \times 8 + c$$

A binary vector `pieces_used` tracks which of the three pieces have been placed in the current round. A `combo` counter is incremented by the number of lines cleared in each placement, and reset to zero when a full round completes without any clearing.

The environment exposes `get_t_plus_3_candidates(`γ`)`, which enumerates all 3! permutations of placement orders and all valid positions, returning the terminal board, cumulative discounted reward, and action sequence for each candidate. This is used by the DVN planner (Section **??**).

2.4 Reward Shaping

Reward design was iterative. The final reward function for both variants is built around two key incentives: survival and line clearing.

2.4.1 Initial Quadratic Reward (Abandoned)

In early experiments, we used a quadratic bonus for clearing multiple lines simultaneously:

$$r = b \cdot n^2$$

where n is the number of lines (rows + columns) cleared in a single move and $b = 10$. The intuition was to strongly incentivise combos. However, this led to poor convergence: the agent received near-zero reward for the vast majority of steps (since clearing multiple lines at once is rare early in training), making the reward signal extremely sparse.

2.4.2 Final Reward Function (1P)

The final 1P reward is computed for each hypothetical afterstate:

$$r = r_{\text{survive}} + b \cdot n_{\text{cleared}} + b \cdot \rho - p_{\text{no3x3}} \tag{1}$$

where:

- $r_{\text{survive}} = 5.0$: a small bonus for not dying,
- $b = 10$, n_{cleared}: number of full rows/columns cleared,
- $\rho \in [0, 1]$: *emptiness ratio* of the resulting board (fraction of empty cells), rewarding the agent for keeping the board clean,
- $p_{\text{no3x3}} = 20.0$ if no 3×3 window of empty cells exists after placement (a proxy for "the board is getting too full").

Invalid moves (placement outside valid mask) incur a penalty of -100 and terminate the episode.

2.4.3 Final Reward Function (3P)

The 3P variant uses a simpler combo-based reward:

$$r = \begin{cases} 0.1 & \text{if no lines cleared} \\ b \cdot (n_{\text{cleared}}^2 + \text{combo}) & \text{if } n_{\text{cleared}} > 0 \end{cases} \tag{2}$$

The `combo` counter accumulates across placements within and across rounds, encouraging the agent to maintain sustained clearing streaks.

3 DQN-Based Approaches

3.1 Architecture

Both DDQN and Rainbow share the same convolutional backbone, `BlockBlastCNNNet1P`, which takes as input the board, the current piece, and the valid placement mask:

1. **Board branch**: two channels (board + valid mask), $8 \times 8 \to$ Conv2d($2 \to 32$, $k = 3$) $\to$ Conv2d($32 \to 64$, $k = 3$) $\to$ Flatten $\to$ 4096 features.

2. **Piece branch**: one channel $(5 \times 5) \to$ Conv2d$(1 \to 32,\ k = 3) \to$ Flatten $\to$ 800 features.

3. **FC head**: (4096+800) $\to$ Linear(4896 $\to$ 512) $\to$ Linear(512 $\to$ 256) $\to$ Linear(256 $\to$ 64) Q-values.

Total parameters: **2,674,464**.

3.2 Double DQN (DDQN)

Double DQN [**?**] decouples action selection from action evaluation in the target:

$$y_t = r_t + \gamma \cdot Q_{\theta^-}(s_{t+1},\ \arg\max_a Q_\theta(s_{t+1}, a))$$

where θ is the policy network and θ^- the target network (updated every 10 episodes). Invalid actions are masked with a large negative value (-10^9) before taking the argmax.

Training hyperparameters: ε decayed from 1.0 to 0.05 ($\times$0.9995/episode), replay buffer of 10,000 transitions, batch size 64, $\gamma = 0.99$, Adam optimizer with lr $= 10^{-4}$.

3.3 Rainbow DQN

Rainbow [**?**] combines five extensions into a single agent:

- **Distributional RL (C51)**: the network outputs a probability distribution over 51 atoms in $[v_{\min} = -10, v_{\max} = 10]$ instead of a scalar. The loss is the KL divergence between the projected target distribution and the current prediction.

- **Prioritized Experience Replay (PER)**: transitions are sampled with probability proportional to $|\delta|^\alpha$ (priority exponent $\alpha = 0.5$), with importance-sampling correction ($\beta = 0.4$).

- **N-step returns**: 3-step bootstrapped targets replace 1-step TD.

- **Noisy Networks**: NoisyLinear layers with factorised Gaussian noise ($\sigma_0 = 0.5$) replace ε-greedy exploration.

- **Dueling architecture**: separate value and advantage streams, recombined as $Q = V + A - \bar{A}$.

3.4 Limitations of DQN Approaches

Despite 1,000 training episodes, neither DDQN nor Rainbow achieved consistent long-game performance. We attribute this to several structural difficulties:

1. **Credit assignment**: a placement that sets up a line clear two or three moves later receives no immediate reward. With $\gamma = 0.99$ and episode lengths of 20–30, the discounted credit is weak.

2. **Q-function instability**: with an action space of 64 outputs, the network must simultaneously learn to distinguish all valid placements. The valid mask varies drastically between steps, making the Q-value landscape highly non-stationary.

3. **Sample efficiency**: the 2.67M parameter model requires far more data than the 1,000-episode budget. The DQN is essentially fitting a large model to a very small dataset.

4. **Sparse reward signal**: most steps have no line clears. The survival and emptiness bonuses help, but the network struggles to back-propagate meaningful gradients through many zero-reward transitions.

In essence, DDQN and Rainbow treat this as a raw perception problem, forcing the network to implicitly learn piece-grid interaction patterns from scratch. The DVN, described next, sidesteps this by leveraging the environment's ability to pre-compute all afterstates.

4 Deep Value Network (DVN)

4.1 Afterstate Formulation

The central idea of the DVN is to reformulate action selection. Instead of learning $Q(s, a)$, we learn a *state value function* $V(s')$ where s' is the *afterstate*: the board resulting from placing piece a at position (r, c) and resolving line clears.

Since the environment precomputes `placements_result`, the agent can enumerate all valid afterstates $\{s'_a\}$ and their immediate rewards $\{r_a\}$ at each step, then select:

$$a^* = \arg\max_a \left[r_a + \gamma \cdot V_\theta(s'_a)\right]$$

This is a *one-step lookahead with a learned value function.* The advantages over pure Q-learning are:

- The network only needs to evaluate a board, not a (board, action) pair. This is a simpler function to learn.
- The agent automatically respects valid placement constraints—no masking needed.
- The value function captures long-term board quality (compactness, clearing potential) independently of the current piece.

4.2 Architecture: Multi-Kernel CNN

The value network `BlockBlastValueNet1Pmultikernelflattenned` uses a bank of parallel convolutional branches, each specialised for a different spatial scale.

4.2.1 Convolution Branches

The input is a single 8×8 binary board. Eight parallel branches process it simultaneously:

Branch	Kernel	Filters	Padding	Output spatial
Square 1×1	$(1, 1)$	1	0	8×8
Square 2×2	$(2, 2)$	8	1	9×9
Square 3×3	$(3, 3)$	16	1	8×8
Square 4×4	$(4, 4)$	16	2	9×9
Square 5×5	$(5, 5)$	16	2	8×8
Square 8×8	$(8, 8)$	64	0	1×1
Horizontal 1×8	$(1, 8)$	4	0	8×1
Vertical 8×1	$(8, 1)$	4	0	1×8

The padding uses `ConstantPad2d` with value **1.0** (i.e., the kernel "sees" walls of filled cells outside the board boundary), which encodes the physical constraint that nothing can be placed outside the grid. No padding is applied to the line kernels.

Each branch then applies: Flatten $\rightarrow$ Linear($\cdot \rightarrow 16$) $\rightarrow$ LeakyReLU $\rightarrow$ Linear($16 \rightarrow 8$). All eight branch outputs (8 features each) are concatenated into a **64-dimensional** feature vector.

4.2.2 Classification of Branches

- 1×1: per-cell occupancy features.
- 2×2, 3×3: local block patterns matching common piece shapes.
- 4×4, 5×5: medium-scale structural features.
- 8×8: global board summary (single activation).
- 1×8, 8×1: full-row and full-column detectors—crucial for identifying nearly-complete lines.

4.2.3 MLP Head

The 64-dimensional feature vector is passed through:

$$\text{Linear}(64 \rightarrow 64) \rightarrow \text{LeakyReLU} \rightarrow \text{Linear}(64 \rightarrow 16) \rightarrow \text{LeakyReLU} \rightarrow \text{Linear}(16 \rightarrow 1)$$

The scalar output is $V_\theta(s')$.

Total parameters: **42,461**.

4.3 Training

The DVN is trained via *temporal difference learning on afterstates*. Let $\mathbf{s}'_t$ be the afterstate chosen at time t, r_t the immediate reward, and $\mathbf{s}'_{t+1}$ be the best afterstate reachable at $t + 1$.

TD Target. For non-terminal transitions:

$$y_t = \max_{a'} \left[r_{t+1,a'} + \gamma \cdot V_{\theta^-}(s'_{t+1,a'}) \right]$$

where θ^- is a *target network* (copy of the policy network, updated every 200 episodes). For terminal transitions (game over or invalid move):

$$y_t = \frac{p_{\text{invalid}}}{\gamma}$$

Loss. Huber loss (SmoothL1) between $V_\theta(s'_t)$ and y_t, with gradient clipping at norm 10.

Training protocol.

- 10,000 episodes, max 100 steps per episode.
- ε-greedy: ε from 1.0 to 0.01, decay 0.999/episode.
- Replay buffer: 100,000 transitions, batch size 128.
- Adam optimizer, lr $= 10^{-4}$, StepLR scheduler ($\gamma_{\text{sched}} = 0.99$, step 1000).
- Checkpoints saved every 500 episodes with full training state (RNG, optimizer, scheduler) for exact resume.
- Logged via Weights & Biases.

4.4 3P Planner

For the 3P variant, we extend the DVN with a *round planner* (`RoundPlanner3P`). At the start of each round, it calls `get_t_plus_3_candidates(`γ`)` to enumerate all valid 3-action sequences and their cumulative discounted rewards. Each candidate terminal board is scored:

$$\text{score} = R_{t+1:t+3} + \gamma^3 \cdot V_\theta(s_{t+3})$$

The planner selects the sequence with the highest score and executes actions one by one. If an action becomes invalid (due to a board state discrepancy), the plan is rebuilt online.

4.5 Results

4.5.1 1P Benchmark

Figure **??** shows the episode return and length distributions for the trained DVN, a greedy baseline, and a random baseline over 200 evaluation episodes.

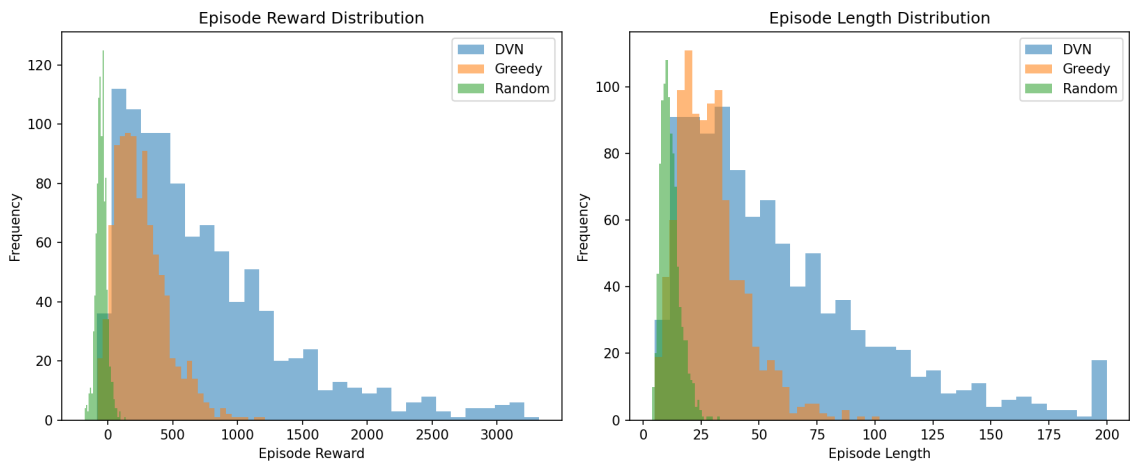

Figure 1: Distribution of episode returns (left) and episode lengths (right) for DVN, greedy, and random agents on the 1P Block Blast environment.

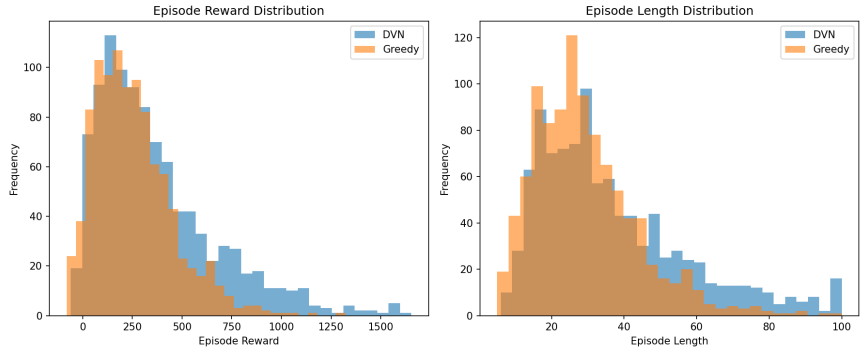

Figure 2: DVN vs. greedy agent comparison (earlier benchmark run, before random baseline was added).

Key observations:

- The trained DVN achieves an **average of ∼60 steps** per episode, compared to ∼20 steps for an untrained model (pure exploration).
- The greedy baseline (always pick the highest immediate reward placement) performs reasonably but is shortsighted—it tends to over-fill the board without setting up line clears.
- The DVN outperforms both baselines in terms of episode length, demonstrating that the value function captures some long-term structure of the board.

Agent	Avg. Return	Avg. Steps	Parameters
Random	—	∼5	—
Greedy	—	∼30	—
DVN (untrained)	−815.98	19.63	42,461
DVN (trained, 60 avg)	—	∼60	42,461
DDQN 1P	—	∼20	2,674,464
Rainbow 1P	—	∼20	2,674,464

Table 1: Summary of agent performance on the 1P environment.

4.5.2 3P Planner Performance

The 3P planner evaluation was computationally expensive due to the exhaustive 3! permutation enumeration in pure Python/NumPy. Full evaluation was not completed; however, individual game traces confirmed correct plan selection and re-planning behavior when a cached action became invalid.

5 Discussion and Future Work

Why DVN outperforms DQN here. The core advantage of the DVN is that it decouples *what the board looks like after my move* from *which action achieves that board*. Since the environment can precompute all afterstates, the value network only needs to learn a mapping from board states to long-term values—a significantly simpler function

than the Q-function, which must also implicitly encode the current piece and the placement geometry.

Limitations of the current DVN.

- The value function sees only the afterstate board, *not* the upcoming pieces. A board with an isolated 1×1 hole has different long-term value depending on whether a dot piece is available.
- The 3P planner's full enumeration is $O(3! \times 64^3)$ in the worst case—prohibitively slow without batched GPU acceleration.

Future directions.

- **Include next-piece information** in the value network input (additional channels or a separate branch).
- **Hand-crafted features** (number of holes, row/column fill density, height variance) as an auxiliary input branch—these are known to be highly informative in Tetris-like games.
- **Batch the 3P lookahead** on GPU to make the planner tractable for online evaluation.
- **Monte Carlo Tree Search** on top of the value function for deeper planning.

6 Conclusion

We have presented a reinforcement learning study of Block Blast, a puzzle game that challenges agents with long-horizon planning, sparse rewards, and a large combinatorial action space. Standard DQN approaches (DDQN, Rainbow) with $\sim$2.67M parameters struggled to converge meaningfully within the training budget. A *Deep Value Network* operating on afterstates—with only 42,461 parameters and a multi-kernel CNN architecture—achieved substantially better episode lengths and generalised to a three-piece planning setting via exhaustive one-round lookahead. The key insight is that leveraging environment structure (precomputed afterstates) allows the learning problem to be reduced to a simpler value estimation task, making learning more sample-efficient.

A Model Architectures

A.1 BlockBlastCNNNet1P (DDQN / Rainbow)

```
Board + Valid Mask (2, 8, 8)
      Conv2d(2  32  , k=3, pad=1)     ReLU
      Conv2d(32  64  , k=3, pad=1)     ReLU
      Flatten     4096

Piece (1, 5, 5)
      Conv2d(1  32  , k=3, pad=1)     ReLU
      Flatten     800

Concat [4096 + 800] = 4896
      Linear(4896  512  )     ReLU
      Linear(512  256  )     ReLU
      Linear(256  64  )  [Q-values]
```

A.2 BlockBlastValueNet1PmultikernelFlattenned (DVN)

```
Board (1, 8, 8)     8 parallel branches:

Branch k: ConstantPad2d(pad_k, val=1.0)
           Conv2d(1  filters_k  , kernel_k)
           LeakyReLU
           Flatten
           Linear(flat_k     16)     LeakyReLU
           Linear(16     8)

Concat [8 branches    8] = 64
      Linear(64  64  )     LeakyReLU
      Linear(64  16  )     LeakyReLU
      Linear(16  1  )  [V(s')]
```